

SQL CRIB SHEET

Tips, Patterns and Examples for Beginners

Shrewsbury College Context

This crib sheet is here to help you **understand how to write SQL**, not to give you answers. You must always adapt table names, column names, and values to match the task you are working on.

Before You Write Any SQL

Always stop and think first.

Ask yourself:

1. What table has the data I need?
2. What columns do I need?
3. Am I reading data or changing data?
4. Do I need to link tables using IDs?

If you rush straight into typing SQL, mistakes are very easy to make.

DML – Working With Data

(Data Manipulation Language)

You will use these commands the most.

SELECT – Reading Data

Start simple

If you are unsure what a table looks like, always start by selecting everything.

```
SELECT *  
FROM Students;
```

This helps you:

- Check column names
- Check data values
- Avoid spelling mistakes later

Selecting specific columns

Only select the columns the task asks for.

```
SELECT StudentName, Course  
FROM Students;
```

Selecting extra columns can lose marks or cause confusion.

Filtering data using WHERE

Use WHERE to choose which rows you want.

```
SELECT StudentName, YearGroup  
FROM Students  
WHERE YearGroup = 2;
```

Important rules:

- Text values must be inside quotes
- Numbers do not use quotes

Ordering results

Use ORDER BY to sort your results.

```
SELECT StudentName, Course
FROM Students
ORDER BY StudentName ASC;
```

Remember:

- ASC means A to Z
- DESC means Z to A
- If you do not write ASC or DESC, SQL uses ASC by default

Working with dates

Dates in this database are stored as text.

```
SELECT StudentName, EnrolmentDate
FROM Students
WHERE EnrolmentDate < '2023-09-01';
```

Always copy the date format used in the table exactly.

JOIN – Linking Tables Together

Think before you join

Before writing a join, ask:

Which columns link these tables together?

You should always join using ID columns.

Simple join example

```
SELECT Students.StudentName, Courses.CourseName
FROM Students, Courses
WHERE Students.CourseID = Courses.CourseID;
```

Key points:

- Use table names before column names
- Join using matching ID fields
- Never join using names or text

Joining through a junction table

Some relationships need a middle table.

Example logic:

Students → Enrolments → Courses

```
SELECT Students.StudentName, Courses.CourseName
FROM Students, Enrolments, Courses
WHERE Students.StudentID = Enrolments.StudentID
AND Courses.CourseID = Enrolments.CourseID;
```

Always follow the IDs step by step.

INSERT INTO – Adding Data

Adding a new record

```
INSERT INTO Students (StudentID, StudentName, College, YearGroup)
VALUES (101, 'Alex Turner', 'Shrewsbury College', 2);
```

Important rules:

- Always list the column names
- Values must match the column order
- Text values must use quotes

Inserting into a junction table

```
INSERT INTO Enrolments (StudentID, CourseID, StartDate)
VALUES (101, 12, '2024-09-01');
```

Both ID values must already exist or the insert will fail.

UPDATE – Changing Data

⚠ Be very careful with UPDATE

Updating one value

```
UPDATE Students
SET YearGroup = 3
WHERE StudentID = 101;
```

Golden rule:

Never use UPDATE without a WHERE clause.

Updating multiple columns

```
UPDATE Students
SET College = 'Shrewsbury College', YearGroup = 2
WHERE StudentName = 'Alex Turner';
```

Use commas to separate each column change.

Safety check (strongly recommended)

Before running UPDATE, test your WHERE condition first:

```
SELECT *  
FROM Students  
WHERE StudentName = 'Alex Turner';
```

If this shows the correct row, then change SELECT to UPDATE.

DELETE – Removing Data

⚠ High risk command

Deleting one record

```
DELETE FROM Students  
WHERE StudentID = 101;
```

Always target rows carefully.

Deleting from a junction table

```
DELETE FROM Enrolments  
WHERE StudentID = 101  
AND CourseID = 12;
```

Use both IDs to avoid deleting too much.

Deleting multiple records

```
DELETE FROM Students  
WHERE YearGroup < 2;
```

Always test this with SELECT first.

DDL – Working With Table Structure

(Data Definition Language)

CREATE TABLE

Used to create a new table.

```
CREATE TABLE Clubs (  
    ClubID INTEGER PRIMARY KEY,  
    ClubName TEXT NOT NULL,  
    College TEXT  
);
```

Important ideas:

- PRIMARY KEY uniquely identifies each row
- NOT NULL means the value must be provided

ALTER TABLE – Changing a Table

Adding a new column

```
ALTER TABLE Clubs  
ADD Founded TEXT;
```

This is used when requirements change.

DROP TABLE

⚠ Very dangerous

```
DROP TABLE Clubs;
```

This deletes the table and all data permanently.

Only use this if you are completely sure.

Common Mistakes to Avoid

- Misspelling table or column names
- Forgetting quotation marks for text
- Using UPDATE or DELETE without WHERE
- Joining tables using names instead of IDs
- Trying to do everything in one step

Final Student Checklist

Before you run your SQL, ask:

- Am I using the correct table?
- Are my column names spelled correctly?
- Do my text values use quotes?
- Have I used WHERE where needed?
- Have I selected only what the task asked for?

If you can answer yes to all of these, you are working like a confident SQL user.